Experiment No.: 7

Title: Usability Testing

Batch: B–1 **Roll No.: 16010422234** **Experiment No.: 7**

Aim: To perform usability testing using any open source tool.

Resources needed: Internet, Chrome Browser

Theory:

Usability testing, also known as User Experience (UX) testing, is a testing method for measuring how easy and user-friendly a software application is. A small set of target end-users, use software applications to expose usability defects. Usability testing mainly focuses on the user's ease of using application, flexibility of application to handle controls and ability of application to meet its objectives. This testing is recommended during the initial design phase of SDLC, which gives more visibility on the expectations of the users.

Users are asked to complete tasks, typically while they are being observed by a researcher, to see where they encounter problems and experience confusion. If more people encounter similar problems, recommendations will be made to overcome these usability issues.

“Usability engineering” is the process of identifying users’ needs to ensure a product can achieve specific goals effectively and efficiently, which results in overall satisfaction and success. To mitigate user issues and ensure your product is working for its intended audience, you need to venture into the world of usability engineering.

Usability is a combination of many factors and theories, but all can be targeted to achieve the goals below:

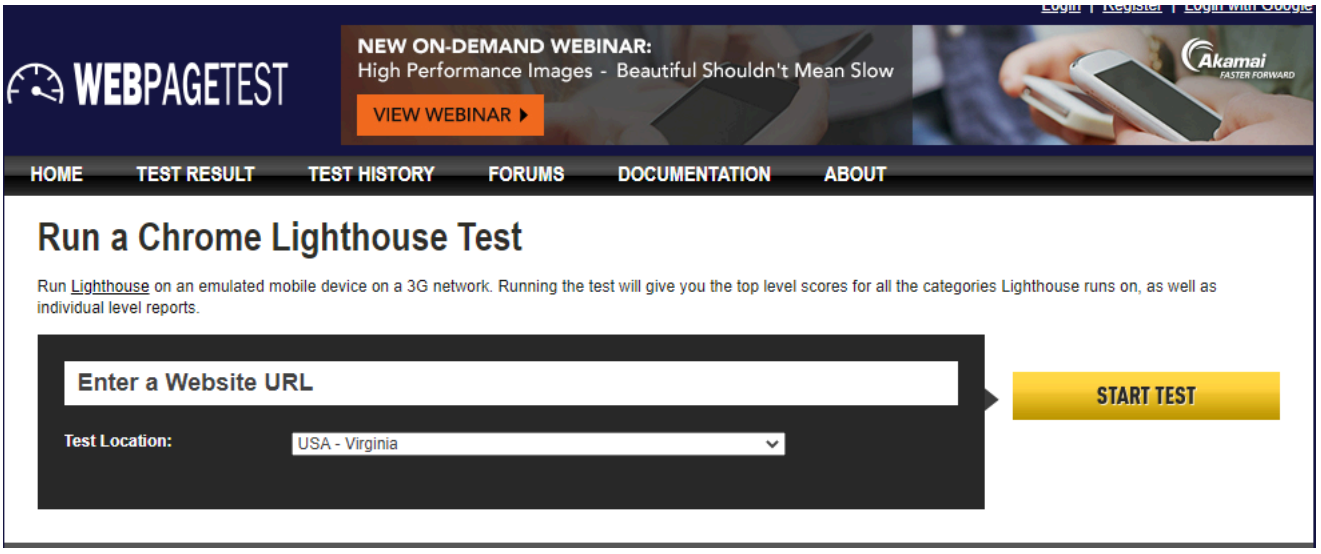
- Intuitive Design and/or user interface
- Ease of learning
- Efficiency of use
- Memorability
- Error prevention
- User Satisfaction

It is important for any web designer to design functional sites that can easily generate interest and online traffic among the internet users. The **user interface design** of your site plays a vital role in bringing high volume web traffic to it. Therefore, web designers should give due importance to the **user interface** of the site which is being designed by them. In the present era, there is an explosive growth in e-commerce with billions of dollars worth of sales taking place each year. Today, thousands of businesses are completely dependent on the internet for their success.

The Internet has emerged as a one-stop source of entertainment, information, social interaction and commerce. For success in any online business, a user friendly website is a must as it will provide enhanced user experience to the online visitors. Any site that is too complex and difficult will definitely push away online traffic. The use of effective and simple user interface design will be of immense help in achieving the objectives of a website. A good user interface not only increases the site usability but also leads to the smooth completion of any task at hand thereby making everything enjoyable and flexible as per the requirements of users.

Following are the tools which are useful in usability testing:

- Lighthouse
- Website Grader
- Google PageSpeed Tools
- GTmetrix

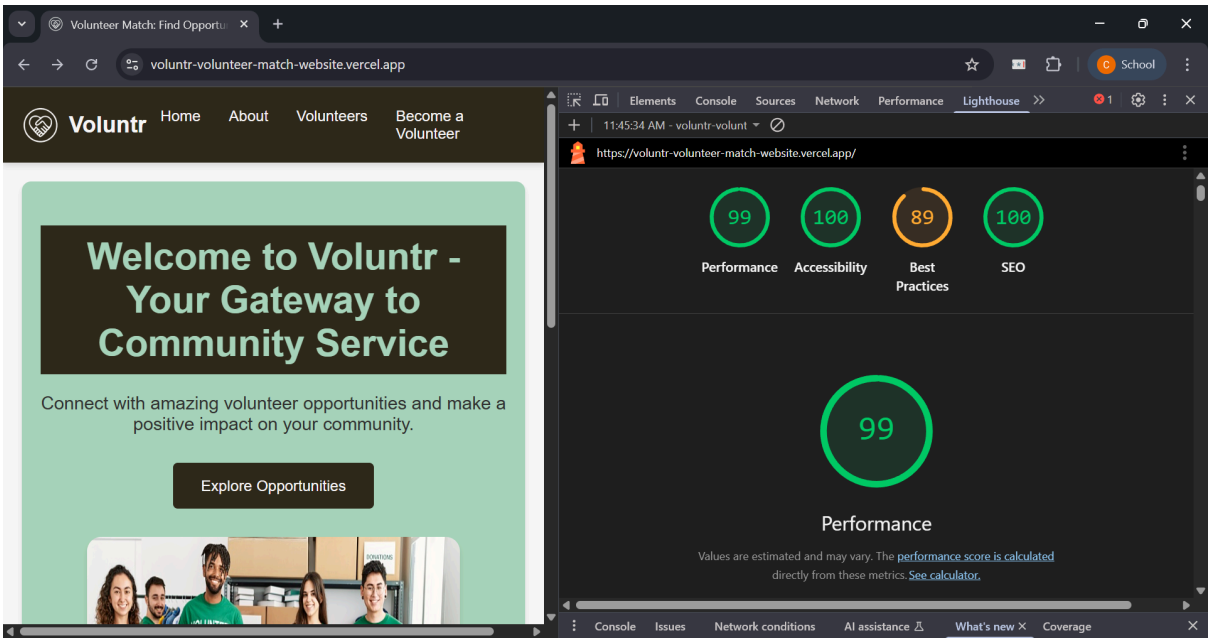


Webpagetest: Lighthouse

Procedure:

1. **Explore any one tool for usability testing.**

   The open-source tool selected and utilized for this usability testing experiment is **Lighthouse**. Lighthouse is an automated auditing tool developed by Google that assesses the quality of web pages across five key categories: Performance, Accessibility, Best Practices, Search Engine Optimization (SEO), and Progressive Web App (PWA) capabilities. It provides actionable feedback to ensure web products are fast, accessible, and follow modern web standards.



2. **Perform usability testing of any website and generate the report for the same.**

   Website Tested: https://voluntr-volunteer-match-website.vercel.app/

   The Lighthouse audit was conducted using the Chromium DevTools under a desktop emulation and custom throttling.

3. **Analyze and summarize reports for quality and performance of web pages.**

   Performance Analysis (Score: 99/100)
   The website demonstrates exceptional loading speed and responsiveness. Key performance metrics (Core Web Vitals) are as follows:

   - **First Contentful Paint (FCP):** 0.2 s.
   - **Largest Contentful Paint (LCP):** 0.8 s. The LCP element is the main heading: "Welcome to Voluntr - Your Gateway to Community Service".
   - **Total Blocking Time (TBT):** 0 ms.
   - **Cumulative Layout Shift (CLS):** 0.

   These metrics indicate that content loads nearly instantly, and the user interface is completely stable with no input delay, providing a superior user experience.

   Accessibility Analysis (Score: 100/100)
   The perfect score confirms that the site adheres to all automated accessibility checks, promoting inclusivity.

   - **Color Contrast:** Background and foreground colors have a sufficient contrast ratio.
   - **Semantic Structure:** Buttons have accessible names , image elements have [alt] attributes , and heading elements appear in sequentially-descending order.
   - **Viewport:** The viewport meta tag is correctly configured, which is crucial for mobile optimization and preventing input delay.

Best Practices Analysis (Score: 89/100)
The minor drop in score is attributed to critical security and maintenance issues:

- **Security Vulnerabilities:** The audit failed checks for effective security policies, specifically noting "No CSP found in enforcement mode", "No COOP header found", and "No frame control policy found" (for X-Frame-Options/CSP). These missing headers expose the application to high-severity attacks like XSS and clickjacking.
- **Image Sizing:** A significant finding under best practices is that an image file (logo.89ffbec....jpg) is larger than its displayed dimensions (606x605 vs. 60x60), wasting bandwidth.
- **Browser Errors:** Unresolved problems were logged to the console, including a syntax error in the manifest.json file.

SEO Analysis (Score: 100/100)
The site is highly optimized for search engine discoverability. Key factors include:

- The page is not blocked from indexing.
- The document has a <title> element and a meta description.
- Links are crawlable and have descriptive text.

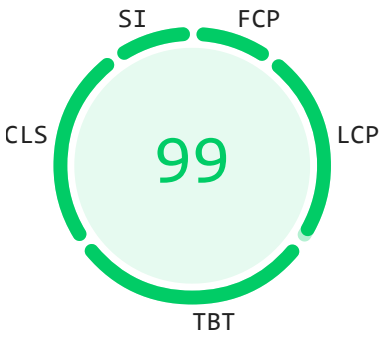
4. Suggest improvements based on your analysis.

Based on the audit, the primary focus areas should be security hardening and content optimization:

1. **Mandatory Security Hardening:** Immediately implement the required security headers, specifically:
    a. **Content Security Policy (CSP):** To prevent Cross-Site Scripting (XSS) attacks.
    b. **Cross-Origin-Opener-Policy (COOP):** To ensure proper origin isolation.
    c. **X-Frame-Options (XFO) or CSP frame-ancestors:** To mitigate clickjacking.
2. **Optimize Image Assets:** Rectify the inefficient image delivery flagged as a major saving opportunity (Est savings of 580 KiB).
    a. Serve images, particularly the logo, at their actual display size to reduce file size.
    b. Consider using modern image formats (WebP/AVIF) and applying efficient cache lifetimes to reduce repeat-visit load times.
3. **Third-Party Code Management:** Evaluate the necessity and loading strategy for the Google Tag Manager script, as it is a major contributor to main-thread activity and transfer size. Consider lazy-loading or deferring its execution.
4. **Resolve Maintenance Issues:** Fix the "Syntax error" in the manifest.json file to address logged browser errors.

https://voluntr-volunteer-match-website.vercel.app/

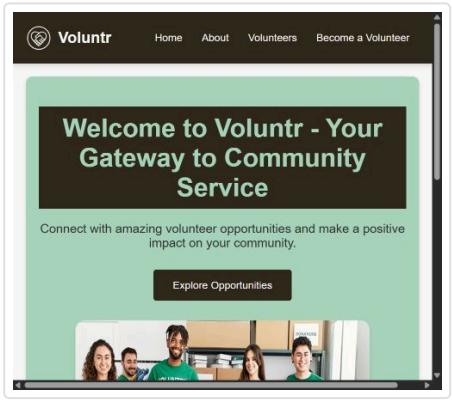
Performance	Accessibility	Best Practices	SEO
99	100	89	100



Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

▲ 0–49 ■ 50–89 ● 90–100



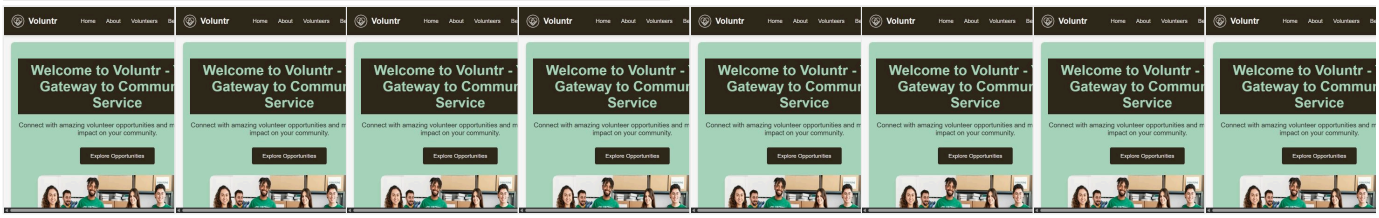
METRICS

Expand view

- First Contentful Paint: 0.2 s
- Largest Contentful Paint: 0.8 s
- Total Blocking Time: 0 ms
- Cumulative Layout Shift: 0

Speed Index

0.3 s



Later this year, insights will replace performance audits. Learn more and provide feedback here.

Go back to audits

Show audits relevant to: All FCP LCP TBT

INSIGHTS

Use efficient cache lifetimes — Est savings of 677 KiB

A long cache lifetime can speed up repeat visits to your page. Learn more. FCP LCP

Request	Cache TTL	Transfer Size
vercel.app 1st Party		677 KiB
...media/logo.89ffbec....jpg (voluntr-volunteer-match-website.vercel.app)	None	510 KiB
...media/hero-image.958326d....jpg (voluntr-volunteer-match-website.vercel.app)	None	99 KiB
...js/main.fc6759a5.js (voluntr-volunteer-match-website.vercel.app)	None	66 KiB
...css/main.3ffd731d.css (voluntr-volunteer-match-website.vercel.app)	None	1 KiB

Improve image delivery — Est savings of 580 KiB

Reducing the download time of images can improve the perceived load time of the page and LCP. Learn more about optimizing image size FCP LCP

URL	Resource Size	Est Savings
vercel.app 1st Party	608.6 KiB	580.0 KiB
...media/logo.89ffbec....jpg (voluntr-volunteer-match-website.vercel.app)	509.9 KiB	509.3 KiB
Using a modern image format (WebP, AVIF) or increasing the image compression could improve this image's download size.		450.3 KiB
This image file is larger than it needs to be (606x605) for its displayed dimensions (60x60). Use responsive images to reduce the image download size.		504.9 KiB

URL	Resource Size	Est Savings
...media/hero-image.958326d....jpg (voluntr-volunteer-match-website.vercel.app)	98.7 KiB	70.6 KiB
Using a modern image format (WebP, AVIF) or increasing the image compression could improve this image's download size.		70.6 KiB

▲ Network dependency tree

Avoid chaining critical requests by reducing the length of chains, reducing the download size of resources, or deferring the download of unnecessary resources to improve page load. LCP

Maximum critical path latency: **95 ms**

Initial Navigation

- https://voluntr-volunteer-match-website.vercel.app - **60 ms,** 1.09 KiB
  - ...css/main.3ffd731d.css (voluntr-volunteer-match-website.vercel.app) - **95 ms,** 1.32 KiB

Preconnected origins

preconnect hints help the browser establish a connection earlier in the page load, saving time when the first request for that origin is made. The following are the origins that the page preconnected to.

Origin	Source
https://www.googletagmanager.com/	link

Preconnect candidates

Add preconnect hints to your most important origins, but try to use no more than 4.

No additional origins are good candidates for preconnecting

■ Render blocking requests

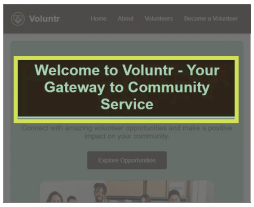
Requests are blocking the page's initial render, which may delay LCP. Deferring or inlining can move these network requests out of the critical path. FCP LCP

URL	Transfer Size	Duration
vercel.app 1st Party	**1.3 KiB**	**0 ms**
...css/main.3ffd731d.css (voluntr-volunteer-match-website.vercel.app)	1.3 KiB	

LCP breakdown

Each subpart has specific improvement strategies. Ideally, most of the LCP time should be spent on loading the resources, not within delays. LCP

Subpart	Duration
Time to first byte	30 ms
Element render delay	230 ms



h1

3rd parties

3rd party code can significantly impact load performance. Reduce and defer loading of 3rd party code to prioritize your page's content.

3rd party	Transfer size	Main thread time
Google Tag Manager Tag-Manager	**141 KiB**	**96 ms**
/gtag/js?id=G-BJV5RV5RPL (www.googletagmanager.com)	141 KiB	96 ms
noondiphcddnnabmjcihcjfbhfklnnep	**0 KiB**	**11 ms**
chrome-extension://noondiphcddnnabmjcihcjfbhfklnnep/content_script_compiled.js	0 KiB	11 ms

These insights are also available in the Chrome DevTools Performance Panel - record a trace to view more detailed information.

DIAGNOSTICS

Reduce unused JavaScript — Est savings of 84 KiB

Reduce unused JavaScript and defer loading scripts until they are required to decrease bytes consumed by network activity. Learn how to reduce unused JavaScript. FCP LCP

Show 3rd-party resources (1)

URL	Transfer Size	Est Savings
Google Tag Manager Tag-Manager	**131.4 KiB**	**53.3 KiB**
/gtag/js?id=G-BJV5RV5RPL (www.googletagmanager.com)	131.4 KiB	53.3 KiB

URL	Transfer Size	Est Savings
vercel.app 1st Party	**62.8 KiB**	**30.6 KiB**
...js/main.fc6759a5.js (voluntr-volunteer-match-website.vercel.app)	62.8 KiB	30.6 KiB

Avoid long main-thread tasks — 1 long task found

Lists the longest tasks on the main thread, useful for identifying worst contributors to input delay. Learn how to avoid long main-thread tasks TBT

URL	Start Time	Duration
Google Tag Manager Tag-Manager		**52 ms**
/gtag/js?id=G-BJV5RV5RPL (www.googletagmanager.com)	799 ms	52 ms

More information about the performance of your application. These numbers don't directly affect the Performance score.

PASSED AUDITS (25) Hide

Layout shift culprits

Layout shifts occur when elements move absent any user interaction. Investigate the causes of layout shifts, such as elements being added, removed, or their fonts changing as the page loads. CLS

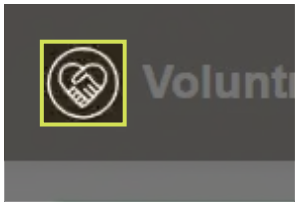
Document request latency

Your first network request is the most important. Reduce its latency by avoiding redirects, ensuring a fast server response, and enabling text compression. FCP LCP

- Avoids redirects
- Server responds quickly (observed 26 ms)
- Applies text compression

Optimize DOM size

A large DOM can increase the duration of style calculations and layout reflows, impacting page responsiveness. A large DOM will also increase memory usage. Learn how to avoid an excessive DOM size.

Statistic	Element	Value
Total elements		25
Most children	 div.home-page	6
DOM depth	 img.logo	6

Duplicated JavaScript

Remove large, duplicate JavaScript modules from bundles to reduce unnecessary bytes consumed by network activity.

Font display

Consider setting `font-display` to `swap` or `optional` to ensure text is consistently visible. `swap` can be further optimized to mitigate layout shifts with font metric overrides.

Forced reflow

A forced reflow occurs when JavaScript queries geometric properties (such as `offsetWidth`) after styles have been invalidated by a change to the DOM state. This can result in poor performance. Learn more about forced reflows and possible mitigations.

INP breakdown

Start investigating with the longest subpart. Delays can be minimized. To reduce processing duration, optimize the main-thread costs, often JS.

LCP request discovery

Optimize LCP by making the LCP image discoverable from the HTML immediately, and avoiding lazy-loading

Legacy JavaScript

Polyfills and transforms enable older browsers to use new JavaScript features. However, many aren't necessary for modern browsers. Consider modifying your JavaScript build process to not transpile Baseline features, unless you know you must support older browsers. Learn why most sites can deploy ES6+ code without transpiling

Modern HTTP

HTTP/2 and HTTP/3 offer many benefits over HTTP/1.1, such as multiplexing. Learn more about using modern HTTP.

Optimize viewport for mobile

Tap interactions may be delayed by up to 300 ms if the viewport is not optimized for mobile.

meta

Defer offscreen images

Consider lazy-loading offscreen and hidden images after all critical resources have finished loading to lower time to interactive. Learn how to defer offscreen images. FCP LCP

Minify CSS

Minifying CSS files can reduce network payload sizes. Learn how to minify CSS. FCP LCP

Minify JavaScript

Minifying JavaScript files can reduce payload sizes and script parse time. Learn how to minify JavaScript. FCP LCP

Reduce unused CSS

Reduce unused rules from stylesheets and defer CSS not used for above-the-fold content to decrease bytes consumed by network activity. Learn how to reduce unused CSS. FCP LCP

Use HTTP/2

HTTP/2 offers many benefits over HTTP/1.1, including binary headers and multiplexing. Learn more about HTTP/2. LCP FCP

Avoid serving legacy JavaScript to modern browsers

Polyfills and transforms enable legacy browsers to use new JavaScript features. However, many aren't necessary for modern browsers. Consider modifying your JavaScript build process to not transpile Baseline features, unless you know you must support legacy browsers. Learn why most sites can deploy ES6+ code without transpiling FCP LCP

Avoids enormous network payloads — Total size was 1,330 KiB

Large network payloads cost users real money and are highly correlated with long load times. Learn how to reduce payload sizes.

☑ Show 3rd-party resources (1)

URL	Transfer Size
vercel.app 1st Party	**1,189.4 KiB**
...media/logo.89ffbec....jpg (voluntr-volunteer-match-website.vercel.app)	510.4 KiB
/favicon.ico (voluntr-volunteer-match-website.vercel.app)	510.4 KiB
...media/hero-image.958326d....jpg (voluntr-volunteer-match-website.vercel.app)	98.9 KiB
...js/main.fc6759a5.js (voluntr-volunteer-match-website.vercel.app)	66.2 KiB
...css/main.3ffd731d.css (voluntr-volunteer-match-website.vercel.app)	1.3 KiB
https://voluntr-volunteer-match-website.vercel.app	1.1 KiB
/manifest.json (voluntr-volunteer-match-website.vercel.app)	1.1 KiB
Google Tag Manager Tag-Manager	**141.1 KiB**
/gtag/js?id=G-BJV5RV5RPL (www.googletagmanager.com)	141.1 KiB

User Timing marks and measures

Consider instrumenting your app with the User Timing API to measure your app's real-world performance during key user experiences. Learn more about User Timing marks.

JavaScript execution time — 0.2 s

Consider reducing the time spent parsing, compiling, and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to reduce Javascript execution time. TBT

Show 3rd-party resources (1)

URL	Total CPU Time	Script Evaluation	Script Parse
vercel.app 1st Party	**160 ms**	**63 ms**	**8 ms**
https://voluntr-volunteer-match-website.vercel.app	104 ms	15 ms	2 ms
...js/main.fc6759a5.js (voluntr-volunteer-match-website.vercel.app)	56 ms	48 ms	6 ms
Google Tag Manager Tag-Manager	**99 ms**	**82 ms**	**15 ms**
/gtag/js?id=G-BJV5RV5RPL (www.googletagmanager.com)	99 ms	82 ms	15 ms

URL	Total CPU Time	Script Evaluation	Script Parse
Unattributable	**68 ms**	**2 ms**	**0 ms**
`Unattributable`	68 ms	2 ms	0 ms

Minimizes main-thread work — 0.3 s

Consider reducing the time spent parsing, compiling and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to minimize main-thread work TBT

Category	Time Spent
Script Evaluation	155 ms
Other	106 ms
Style & Layout	43 ms
Script Parsing & Compilation	23 ms
Parse HTML & CSS	7 ms
Rendering	4 ms

Lazy load third-party resources with facades

Some third-party embeds can be lazy loaded. Consider replacing them with a facade until they are required. Learn how to defer third-parties with a facade. TBT

Uses passive listeners to improve scrolling performance

Consider marking your touch and wheel event listeners as `passive` to improve your page's scroll performance. Learn more about adopting passive event listeners.

Avoids `document.write()`

For users on slow connections, external scripts dynamically injected via `document.write()` can delay page load by tens of seconds. Learn how to avoid document.write().

Page didn't prevent back/forward cache restoration

Many navigations are performed by going back to a previous page, or forwards again. The back/forward cache (bfcache) can

speed up these return navigations. Learn more about the bfcache

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

Hide

- Interactive controls are keyboard focusable

  Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable.

- Interactive elements indicate their purpose and state

  Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints.

- The page has a logical tab order

  Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering.

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering.

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps.

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content.

- HTML5 landmark elements are used to improve navigation

Landmark elements (<main>, <nav>, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements.

- Offscreen content is hidden from assistive technology

Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content.

- Custom controls have associated labels

Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels.

- Custom controls have ARIA roles

Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls.

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (12) Hide

- [aria-hidden="true"] is not present on the document <body>

Assistive technologies, like screen readers, work inconsistently when aria-hidden="true" is set on the document <body>. Learn how aria-hidden affects the document body.

- Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

- Image elements have [alt] attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the alt attribute.

- [user-scalable="no"] is not used in the <meta name="viewport"> element and the [maximum-scale] attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

- Background and foreground colors have a sufficient contrast ratio

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

- Document has a `<title>` element

  The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

- `<html>` element has a `[lang]` attribute

  If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

- `<html>` element has a valid value for its `[lang]` attribute

  Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

- Links have a discernible name

  Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

- Touch targets have sufficient size and spacing.

  Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

- Heading elements appear in a sequentially-descending order

  Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

- Image elements do not have `[alt]` attributes that are redundant text.

  Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute.

NOT APPLICABLE (45)

Hide

- `[accesskey]` values are unique

  Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys.

- `[aria-*]` attributes match their roles

  Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

- Uses ARIA roles only on compatible elements

  Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles.

- `button`, `link`, and `menuitem` elements have accessible names

  When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible.

- ARIA attributes are used as specified for the element's role

  Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

- Deprecated ARIA roles were not used

  Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

- Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

  ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible.

- `[aria-hidden="true"]` elements do not contain focusable descendents

  Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

- ARIA input fields have accessible names

  When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels.

- ARIA `meter` elements have accessible names

  When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `meter` elements.

- ARIA `progressbar` elements have accessible names

  When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements.

- Elements use only permitted ARIA attributes

  Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

- `[role]`s have all required `[aria-*]` attributes

  Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

- Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

  Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements.

- `[role]`s are contained by their required parent element

  Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element.

- `[role]` values are valid

  ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

- Elements with the `role=text` attribute do not have focusable descendents.

  Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute.

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields.

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements.

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements.

- `[aria-*]` attributes have valid values

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

- `[aria-*]` attributes are valid and not misspelled

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks.

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly.

- Definition list items are wrapped in `<dl>` elements

  Definition list items (<dt> and <dd>) must be wrapped in a parent <dl> element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly.

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs.

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels.

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles.

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons.

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text.

- Form elements have associated labels

  Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

- Links are distinguishable without relying on color.

  Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable.

- Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

  Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

- List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

  Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

- The document does not use `<meta http-equiv="refresh">`

  Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag.

- `<object>` elements have alternate text

  Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements.

- Select elements have associated label elements.

  Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element.

- Skip links are focusable.

  Including a skip link can help users skip to the main content to save time. Learn more about skip links.

- No element has a `[tabindex]` value greater than 0

  A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute.

- Tables have different content in the summary attribute and `<caption>`.

  The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption.

- Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

  Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute.

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

  Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers.

- `[lang]` attributes have a valid value

  Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute.

- `<video>` elements contain a `<track>` element with `[kind="captions"]`

  When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions.

Best Practices

USER EXPERIENCE

Displays images with incorrect aspect ratio

Image display dimensions should match natural aspect ratio. Learn more about image aspect ratio.

	URL	Aspect Ratio (Displayed)	Aspect Ratio (Actual)
vercel.app 1st Party			
 img	...media/hero-image.958326d....jpg (voluntr-volunteer-match-website.vercel.app)	455 x 250 (1.82)	641 x 269 (2.38)

Serves images with low resolution

Image natural dimensions should be proportional to the display size and the pixel ratio to maximize image clarity. Learn how to provide responsive images.

	URL	Displayed size	Actual size	Expected size
vercel.app 1st Party				
 img	...media/hero-image.958326d....jpg (voluntr-volunteer-match-website.vercel.app)	455 x 250	641 x 269	683 x 375

GENERAL

Browser errors were logged to the console

Errors logged to the console indicate unresolved problems. They can come from network request failures and other browser concerns. Learn more about this errors in console diagnostic audit

Source	Description
vercel.app 1st Party	
manifest.json:1	`Manifest: Line: 1, column: 1, Syntax error.`
(index):1	`Unchecked runtime.lastError: Could not establish connection. Receiving end does not exist.`

Detected JavaScript libraries

All front-end JavaScript libraries detected on the page. Learn more about this JavaScript library detection diagnostic audit.

Name	Version
Create React App	

TRUST AND SAFETY

Ensure CSP is effective against XSS attacks

A strong Content Security Policy (CSP) significantly reduces the risk of cross-site scripting (XSS) attacks. Learn how to use a CSP to prevent XSS

Description	Directive	Severity
No CSP found in enforcement mode		High

Ensure proper origin isolation with COOP

The Cross-Origin-Opener-Policy (COOP) can be used to isolate the top-level window from other documents such as pop-ups. Learn more about deploying the COOP header.

Description	Directive	Severity
No COOP header found		High

Mitigate clickjacking with XFO or CSP

The `X-Frame-Options` (XFO) header or the `frame-ancestors` directive in the `Content-Security-Policy` (CSP) header control where a page can be embedded. These can mitigate clickjacking attacks by blocking some or all sites from

embedding the page. Learn more about mitigating clickjacking.

Description	Severity
No frame control policy found	High

Mitigate DOM-based XSS with Trusted Types

The `require-trusted-types-for` directive in the `Content-Security-Policy` (CSP) header instructs user agents to control the data passed to DOM XSS sink functions. Learn more about mitigating DOM-based XSS with Trusted Types.

Description	Severity
No `Content-Security-Policy` header with Trusted Types directive found	High

PASSED AUDITS (11) Hide

Uses HTTPS

All sites should be protected with HTTPS, even ones that don't handle sensitive data. This includes avoiding mixed content, where some resources are loaded over HTTP despite the initial request being served over HTTPS. HTTPS prevents intruders from tampering with or passively listening in on the communications between your app and your users, and is a prerequisite for HTTP/2 and many new web platform APIs. Learn more about HTTPS.

Avoids deprecated APIs

Deprecated APIs will eventually be removed from the browser. Learn more about deprecated APIs.

Avoids third-party cookies

Third-party cookies may be blocked in some contexts. Learn more about preparing for third-party cookie restrictions.

Allows users to paste into input fields

Preventing input pasting is a bad practice for the UX, and weakens security by blocking password managers.Learn more about user-friendly input fields.

Avoids requesting the geolocation permission on page load

Users are mistrustful of or confused by sites that request their location without context. Consider tying the request to a user action instead. Learn more about the geolocation permission.

Avoids requesting the notification permission on page load

Users are mistrustful of or confused by sites that request to send notifications without context. Consider tying the request to user gestures instead. Learn more about responsibly getting permission for notifications.

- Has a `<meta name="viewport">` tag with `width` or `initial-scale`

A `<meta name="viewport">` not only optimizes your app for mobile screen sizes, but also prevents a 300 millisecond delay to user input. Learn more about using the viewport meta tag.

- Page has the HTML doctype

Specifying a doctype prevents the browser from switching to quirks-mode. Learn more about the doctype declaration.

- Properly defines charset

A character encoding declaration is required. It can be done with a `<meta>` tag in the first 1024 bytes of the HTML or in the Content-Type HTTP response header. Learn more about declaring the character encoding.

- No issues in the `Issues` panel in Chrome Devtools

Issues logged to the `Issues` panel in Chrome Devtools indicate unresolved problems. They can come from network request failures, insufficient security controls, and other browser concerns. Open up the Issues panel in Chrome DevTools for more details on each issue.

- Page has valid source maps

Source maps translate minified code to the original source code. This helps developers debug in production. In addition, Lighthouse is able to provide further insights. Consider deploying source maps to take advantage of these benefits. Learn more about source maps.

URL	Map URL
vercel.app 1st Party	
...js/main.fc6759a5.js (voluntr-volunteer-match-website.vercel.app)	...js/main.fc6759a5.js.map (voluntr-volunteer-match-website.vercel.app)
`Error: Timed out fetching resource`	

NOT APPLICABLE (3)

Hide

- Redirects HTTP traffic to HTTPS

Make sure that you redirect all HTTP traffic to HTTPS in order to enable secure web features for all your users. Learn more.

- Use a strong HSTS policy

  Deployment of the HSTS header significantly reduces the risk of downgrading HTTP connections and eavesdropping attacks. A rollout in stages, starting with a low max-age is recommended. Learn more about using a strong HSTS policy.

- Document uses legible font sizes

  Font sizes less than 12px are too small to be legible and require mobile visitors to “pinch to zoom” in order to read. Strive to have >60% of page text ≥12px. Learn more about legible font sizes.

SEO

These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not score here that may affect your search ranking, including performance on Core Web Vitals. Learn more about Google Search Essentials.

ADDITIONAL ITEMS TO MANUALLY CHECK (1)

Hide

- Structured data is valid

  Run the Structured Data Testing Tool and the Structured Data Linter to validate structured data. Learn more about Structured Data.

Run these additional validators on your site to check additional SEO best practices.

PASSED AUDITS (9)

Hide

- Page isn’t blocked from indexing

  Search engines are unable to include your pages in search results if they don't have permission to crawl them. Learn more about crawler directives.

- Document has a `<title>` element

  The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

- Document has a meta description

Meta descriptions may be included in search results to concisely summarize page content. Learn more about the meta description.

- Page has successful HTTP status code

Pages with unsuccessful HTTP status codes may not be indexed properly. Learn more about HTTP status codes.

- Links have descriptive text

Descriptive link text helps search engines understand your content. Learn how to make links more accessible.

- Links are crawlable

Search engines may use `href` attributes on links to crawl websites. Ensure that the `href` attribute of anchor elements links to an appropriate destination, so more pages of the site can be discovered. Learn how to make links crawlable

- robots.txt is valid

If your robots.txt file is malformed, crawlers may not be able to understand how you want your website to be crawled or indexed. Learn more about robots.txt.

- Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

- Document has a valid `hreflang`

hreflang links tell search engines what version of a page they should list in search results for a given language or region. Learn more about `hreflang`.

NOT APPLICABLE (1)

Hide

- Document has a valid `rel=canonical`

Canonical links suggest which URL to show in search results. Learn more about canonical links.

Captured at Oct 10, 2025, 11:45 AM GMT+5:30
Initial page load

Emulated Desktop with Lighthouse 12.8.1
Custom throttling

Single page session

Using Chromium 140.0.0.0 with devtools

Questions:

1. **Describe usability and accessibility testing. Explain any five test cases/scenarios for testing usability and accessibility of a web application.**

   Ans: Usability Testing: Usability testing (also known as User Experience (UX) testing) is a testing method that measures how easy, effective, and user-friendly a software application is to use. It involves a small set of target end-users attempting to complete specific tasks while a researcher observes them to identify usability defects. The core focus is on the user's ease of using the application, flexibility to handle controls, and ability to meet its objectives. Usability aims for: Intuitive Design, Ease of Learning, Efficiency of Use, Memorability, Error Prevention, and User Satisfaction.

   Accessibility Testing: Accessibility testing is focused on ensuring that a web application can be used effectively by people with disabilities (such as visual, auditory, cognitive, or motor impairments). The goal is to verify that all users can perceive, operate, and understand the content, typically by adhering to standards like the Web Content Accessibility Guidelines (WCAG).

   Five Test Cases/Scenarios for Usability and Accessibility:

Type	Test Case/Scenario	Focus
Usability	Scenario 1: New User Registration and Login	Ease of Learning & Efficiency
	Goal: A new user successfully creates an account and logs in.	Test Steps: Verify that form fields are clearly labeled, error messages are helpful, and the navigation flow is direct.
Usability	Scenario 2: Primary Goal Completion (e.g., Purchasing an item)	Efficiency of Use & Task Completion
	Goal: A user navigates to the core function of the site and successfully completes a transaction.	Test Steps: Measure the number of clicks and time required. Ensure clear feedback is provided upon task completion.
Accessibility	Scenario 3: Keyboard-Only Navigation	Motor Impairment & Operability

	Goal: A user navigates and interacts with all elements (links, buttons, forms) using only the Tab key and Enter/Space.	Test Steps: Ensure a highly visible focus indicator is present and the tab order is logical (matches the visual layout).
Accessibility	Scenario 4: Screen Reader Compatibility for Images	Visual Impairment & Perceivability
	Goal: A user using a screen reader attempts to understand the images and non-text content.	Test Steps: Verify that all informative image elements have descriptive alt attributes and decorative images have null or empty alt text.
Usability	Scenario 5: Error Handling and Recovery	Error Prevention & Memorability
	Goal: A user intentionally makes a mistake in a form (e.g., wrong password format or missing required field).	Test Steps: Assess if the error message is clear, non-technical, and guides the user on how to fix the error, and if input data is retained.

2. **What are the benefits of usability testing for website owners?**

Ans: Usability testing is vital for website owners because it directly impacts business success, user satisfaction, and operational costs. The benefits include:

- **Increased User Satisfaction and Retention:** An easy-to-use website provides an enhanced user experience (UX), leading to greater satisfaction, reduced frustration, and increased loyalty among online visitors.
- **Higher Conversion Rates:** When a site is intuitive and efficient, users can complete desired tasks—like sign-ups, information retrieval, or purchases—with fewer roadblocks, directly translating to higher sales and goal completion.
- **Reduced Development and Support Costs:**
    - **Early Defect Detection:** Testing is recommended during the initial design phase of the SDLC, allowing developers to address user expectations and fix defects when they are cheapest to correct.
    - **Lower Support Load:** A user-friendly site results in fewer user errors and confusion, leading to fewer customer support inquiries and lower associated operational costs.
- **Increased Online Traffic and Engagement:** A functional, simple, and user-friendly website generates interest and encourages repeat visits, which is essential for success in any online business.
- **Competitive Advantage:** Websites that focus on a seamless and enjoyable user interface design stand out from competitors that are "too complex and difficult".

Outcomes: CO3 – Apply recent automation tools for testing software.

Conclusion: (Conclusion to be based on outcomes)

The usability testing conducted on the Voluntr website using the open-source Lighthouse tool successfully achieved the aim of the experiment. The site demonstrated Excellent Usability and Performance with high scores across most categories: Performance (99/100), Accessibility (100/100), Best Practices (89/100), and SEO (100/100). Specifically, the site exhibited extremely fast loading times, evidenced by key metrics like First Contentful Paint (0.2 s) and Largest Contentful Paint (0.8 s), contributing directly to high user satisfaction and efficiency of use. The Accessibility score of 100 confirms the site's strong adherence to inclusive design principles, while the perfect SEO score ensures high discoverability. However, the analysis of the Best Practices (89/100) section identified critical areas for improvement related to security headers (CSP, COOP, XFO) and image optimization. The experiment demonstrates that continuous usability and performance auditing is essential for maintaining a high-quality, secure, and performant user experience, even for applications that are already highly optimized.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. https://developers.google.com/web/tools/lighthouse/
2. https://developers.google.com/speed/pagespeed/insights/
3. https://www.webpagetest.org/lighthouse
4. https://www.guru99.com/usability-testing-tutorial.html
5. https://gtmetrix.com/
